Towards An Efficient Searching Approach of ROS Message by Knowledge Graph

Sun Bo[1,2], *Xinjun Mao*[1,2*], *Shuo Yang*[1,2], *Long Chen*[1,2]
[1]*College of Computer,National University of Defense Technology, Changsha, China*
[2]*Key Laboratory of Software Engineering for Complex Systems, National University of Defense Technology, Changsha, China*
{sunbo, xjmao, yangshuo11, chen_long} @nudt.edu.cn

2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC) | 978-1-6654-8810-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/COMPSAC54236.2022.00145

***Abstract*—The Robot Operating System (ROS) has become the most popular robot development framework in the last few years, which has loosely coupled structure and provides remote communications between different component nodes. The ROS messages are critical to bridge the communication channels and clearly define the data structures. The developers can use the standardized or user-customized ROS message types to construct a communication channel between two component nodes uniquely. However, it becomes increasingly difficult for developers to find the required ROS message type from thousands of diverse ROS message types in ROS-based robotic software development. Finding the proper ROS message type is a non-trivial task because developers may hardly know the exact names of required ROS messages but only has a rough knowledge of the task domain features. To tackle this challenge, we construct a novel ROS Message Knowledge Graph (RMKG) with 4543 entities and 14320 relationships, including all ROS message types and message packages. We take the shortest path algorithm to search ROS message in RMKG by searching with ROS message feature or ROS message package and visualize the subgraph structure of the search results. Moreover, we develop a ROS message package library that supports fuzzy queries to find the required message package. A comprehensive evaluation of RMKG shows the high accuracy of our knowledge construction approach. A user study indicates that RMKG is promising in helping developers find suitable ROS message types for robotics software development tasks. An effect evaluation of message package fuzzy query shows the good effects of our fuzzy query method under different situations.**

***Index Terms*—search, Knowledge Graph, message, feature, fuzzy query, ROS**

I. Introduction

Nowadays, as autonomous robots are increasingly utilized in human's daily life, robotic software development has been paid much attention both from the academic and industrial developers. In the past few decades, the Robot Operating System (ROS) has become a defacto standard in the robotic community, which enables modular and efficient development of robotic software for a wide range of robotic task domains. The ROS framework is essentially a component-based software development framework that abstracts a specific robotic functionality as an independent component node, and constructs a complete robot software by composing a set of ROS component nodes. In a ROS-based robot software, the most critical thing is to design and implement the communication channels between each connecting component node. ROS message provides the standardized communication interfaces between each ROS node component, and enables easy customization of message types to cover a wide range of development requirements for different robot task domains. A simple example of ROS message type is shown in Fig. 1. The composition of messages in ROS is shown in the official documentation[1]. In general, there are two main ways to compose messages:

```
1  geometry_msgs/msg/Point.msg
2  # This contains the positon of a point in space
3     float x
4     float y
5     float z
```

Fig. 1. Composite data type: Point, representing a 3D coordinate point

- **Basic data types**: ROS built-in basic data types. Standard primitive types (integer, floating-point, boolean, etc.) are supported, as are arrays of primitive types and constants [1].
- **Composite data types**: Messages can be composed of other messages, and arrays of other messages, nested arbitrarily deep [1].

In ROS-based robotic software development, a large portion of development effort has been investigated into the design and implementation of customized ROS message types for communications between ROS node components. However, it becomes increasingly difficult for the developers to utilize the ROS messages when constructing the communication channels. Firstly, there is no dedicated tool to help developers search out a required ROS message type from thousands of diverse ROS message types. The efficiency of searching for a specific and known ROS message via existing searching engines, such as the ROS Wiki and Baidu[2] websites, is

* Corresponding author.

[1]http://wiki.ros.org/msg
[2]https://www.baidu.com/

978-1-6654-8810-5/22/$31.00 ©2022 IEEE
DOI 10.1109/COMPSAC54236.2022.00145

extremely low and may fail to get the required knowledge regarding the message type. Secondly, the ROS Wiki does not provide the function of searching ROS message type, only the function of searching ROS package. The general-purpose search engines, such as Bing and Baidu, are not designed for ROS, so the search result contains much irrelevant content. Thirdly, the search content related to ROS messages mainly comes from the developer's blogs and some Q&A websites, which provides less informative information that is inadequate to support software development. So it is often hard to find the ROS message type we need by the Bing and Baidu search engines. Finally, when developing the software for a new robot task domain, developers may hardly know the exact names of required ROS messages but only has a rough knowledge of the task domain features. It becomes much difficult for developers to search out a required ROS message type based on only roughly estimated keywords of task domains. It is difficult for developers to know which composite message type they need merely based on the task domain features. There is too little information to search the composite message type and the related message package by the general search engine.

To tackle the above challenge, this paper proposes an efficient searching approach specifically for ROS message types based on Knowledge Graph. We analyze 202 user-customized ROS messages from 12 ROS projects manually collected from Github[3], with most stars. We manually collect all 3244 ROS messages and 264 ROS message packages from the ROS official website and construct a novel ROS message knowledge graph(RMKG), with 4543 entities, 14320 relationships, and other information like URL. In RMKG, we abstract and specify the relationships between diverse ROS message types. We take the shortest path algorithm to search ROS message in RMKG by searching with message feature or message package and visualize the subgraph structure of the search results. For getting the ROS message package easily, we build a ROS message package library where we apply the fuzzy query to get the package. With this approach, developers could search out a specific ROS message type via both direct and indirect ways inputting a ROS message package containing the specific ROS message type or the feature word of ROS message type.

We conduct three experiments to evaluate our approach. A comprehensive evaluation of RMKG shows the high accuracy of our knowledge construction approach. A user study indicates that RMKG is promising in helping developers find suitable ROS message types for robotics software development tasks. An effect evaluation of message package fuzzy query shows the good effect of our fuzzy query method under different situations. Our main contributions are summarized as follows:

- We construct a novel ROS Message Knowledge Graph (RMKG) containing 4,543 nodes and 14,320 relationships, including all ROS messages and ROS message packages.
- We construct a message package library that supports message package fuzzy query based on string similarity ranking, including all ROS message packages.
- We evaluate the quality of the key step for ROS message knowledge graph construction, the usefulness of the knowledge graph on ROS message searching, and the effect of our message package fuzzy query method.
- We present a novel approach of utilizing the knowledge graph based on feature information of ROS message for effectively and efficiently searching out the required ROS message, and querying ROS message package based on string similarity ranking.

The rest of the paper is organized as follows. Some related work is reviewed in the next section. In Section III, we analyze ROS message types in 12 ROS projects. In Section IV, we present the details of our approach. We provide the evaluation of our work in Section V. We discuss the validity threats in Section VI. Finally, we conclude our work in Section VII.

II. Related Work

A. Recommendation in Software Development

With the increase of software scale, more researchers focus on code recommendation to help developers.

1) API Recommendation: Wang et al. construct a knowledge graph of Mashup-API to provide API recommendations for Mashup developers [2]. Yin et al. construct an API knowledge graph to help inexperienced developers [3]. Based on statistical learning from fine-grained code changes and the changed context, Nguyen presents a recommendation approach to provide relevant API recommendations for developers [4]. Huang et al. propose a Bi-Information source-based Knowledge recommendation approach to tackle the lexical gap and knowledge gap between the natural language description of the programming task and the API description [5]. Gao et al. provide an API recommendation method for Android Apps development by mining the reusable knowledge from the data resource in App stores [6]. Thung introduces two API recommendation systems in libraries and API methods [7]. By exploiting keyword-API associations from the crowdsourced knowledge of Stack Overflow, Rahman et al. propose a recommendation technique recommending a list of relevant APIs for code search [8]. Liu et al. propose an approach to improve top-1 API recommendation using a ranking-based discriminative approach [9]. Chen et al. propose a novel API recommendation approach by combining structural and textual code information [10]. Yuan et al. present a tool LibraryGuru that can recommend suitable Android APIs for given functionality descriptions [11]. He et al. propose an approach to recommend Python APIs in real-time [12]. Nguyen et al. propose an API function calls and source code snippets recommendation approach based on context-aware collaborative filtering technique [13].

2) Code Recommendation: Alnusair et al. propose a semantic-based source-code recommendation approach for constructing and delivering relevant code snippets [14]. WU

[3]https://github.com/

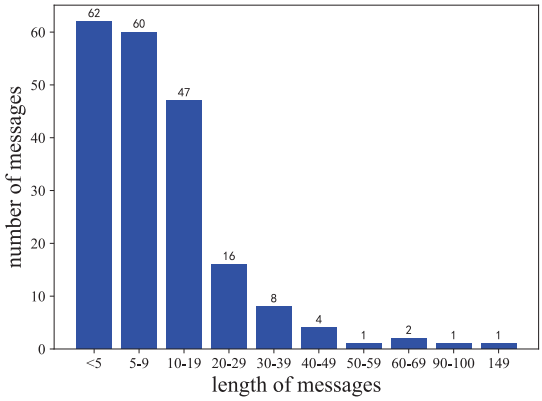

Fig. 2. The message length distribution of 202 messages.

et al. conduct a qualitative review on current code recommendations about Android development and give out two possible improvements of code recommendation to enhance the ability of the state-of-the-art code recommendation techniques and to facilitate the design of code recommendation tools and the tool framework [15]. LUAN et al. propose a code recommendation tool and technique, Aroma, via structural code search, implement Aroma for 4 different languages and develop an IDE plugin for Aroma [16]. Motivated by two findings from a large-scale empirical analysis, Wang et al. propose a context-guided method name recommender [17]. Murakami et al. optimize a search-based code recommendation system with a large-scale code repository and propose an evaluation method for their recommendation system [18].

The above code recommendations are mainly concerned with the traditional software development domain but no research pays attention to robotic code search and recommendation tools to help robot developers, such as ROS developers.

B. Knowledge Graph in Robotics Development

Recently, Knowledge Graph as a form of structured knowledge has become an increasingly popular research direction and has been widely used in industries such as search engines and recommendation systems [19]. Zamanirad et al. built a bot programming platform by constructing an API knowledge graph and applying NLP, ML, and Entity Recognition [20]. Chen et al. designed a ROS package knowledge graph that supports ROS packages search with task description or attributes as input by applying techniques in NLP [21].

To conclude, the utilization of Knowledge Graph has received increasing popularity in robotic software development and other domains. Due to the advantages of rich semantic information and representation of associations between different entities, it is promising to utilize the knowledge graph to search and recommend ROS message types.

III. Motivation and Task Description

To explain the necessity of ROS message searching in robotic software development, we collected 12 projects with the highest number of stars in GitHub labeled ROS and counted the messages in the 12 projects. The statistics show that there are 202 user-customized messages. We refer to the variables defined in a message as message variables and the number of message variables as the length of the message. In general, a message contains more than one message variable. The more message variables, i.e., the longer the message, the more complex the message is. The 202 messages contain a total of 2480 message variables. Fig.2 shows the message length distribution of these 202 messages.

The statistics show that each project contains an average of 16.83 user-customized messages and 12.277 message variables per message. Thus, the number of user-customized messages per project is relatively high, and the average number of message variables per message is more than 12. As can be seen from Fig.2: the percentage of messages less than 5 in length is only 30.69%. The proportion of messages longer than 10 in length is 39.6%. The rate of messages longer than 20 in length is 16.34%. A small number of messages are longer than 50 in length, and some are even 149 in length. So a large percentage of messages in the ROS project is long. In addition, we counted the number of composite message types. Thirty-four messages use the composite message type out of 202 messages, which is only 16.83%. And among the 2480 message variables in 202 messages, the composite message type is 79, only 3.18%. So composite message types are used less frequently, and many messages are composed of more basic data types. The statistics reveal two facts in this paragraph: (1) many ROS messages are long; (2) composite ROS messages are rarely used.

One of the major reasons for this phenomenon is the difficulty for developers to find the composite message types they need. First of all, although the official number of composite message types is 3244 now, many composite message types are not used or known. So developers do not know there are already some composite message types fitting the current task need of setting the messages. They will not search for the related messages but write messages using basic data types. Secondly, when searching in the ROS Wiki, only the package can be searched, but not the message. ROS Wiki does not provide the function of searching for ROS messages directly. However, developers can not directly know the message information contained in the package according to the package name searched. So it is very hard to search for ROS messages by ROS Wiki. Finally, in many cases, when writing a message, the developer only knows the features of the user-customized message. For instance, it is a 3D coordinate point. The features are point, 3D, space, etc. It is difficult and incomplete to find the required message type under the current search engine based only on these features.

Also, since there are currently a total of 3244 officially

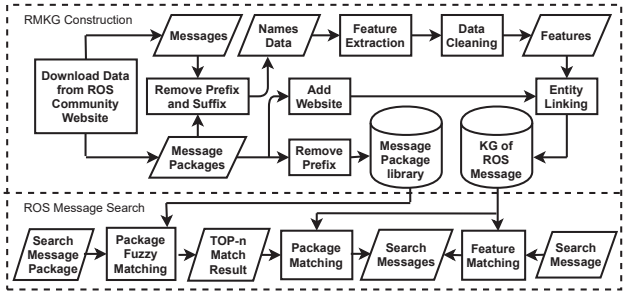

Fig. 3. The overall framework of our approach.

included composite message types[4] on the ROS website, many message types already have reusable identical or similar messages. Even if developers do not need to reuse the existing composite message types exactly, the existing message types relevant to the task can still provide a reference for developers or replicate a part of the message, thus reducing the difficulty of development. In fact, the composite message type provides some semantic information about the message, which can help developers understand the meaning of the message and facilitate later maintenance. The use of the composite message type makes the message more readable and understandable. So it is critically needed for ROS developers to search for the ROS messages, but rather difficult under current search engines to fit the need.

IV. KNOWLEDGE GRAPH-BASED ROS MESSAGE SEARCH APPROACH

This paper proposes a knowledge-graph-based approach to overcome the challenges mentioned above. Knowledge graph provides rich semantic information and deep-level associations between data. Fig.3 shows the key steps of our approach, which contains two parts: mining ROS message knowledge graph from ROS official website and searching ROS message types by message package or the features of message based on the mined knowledge graph[6]. Firstly, we manually collect the information of ROS messages and ROS message packages from the ROS official website[5]. we use the Python module "wordninja"[6] to extract message features from message and message package information after removing the redundant prefixes and suffixes. We manually deal with the data processed by wordninja. Secondly, we build a ROS message knowledge graph called RMKG based on all the above knowledge and other manually added information like URL. Thirdly, We take the shortest path algorithm to search ROS message in RMKGG by searching with message feature or message package and visualize the subgraph structure of the search results. Finally, we collect message package information and use the Python string similarity ranking module "fuzzywuzzy"[7] to build a fuzzy search system of message packages. The developers can search for ROS message types by message package or message features using our approach. The technical details regarding the above steps are illustrated in the following sections.

[4]https://index.ros.org/packages/page/1/time/
[5]https://index.ros.org/packages/page/1/time/
[6]https://pypi.org/project/wordninja/
[7]https://pypi.org/project/fuzzywuzzy/

TABLE I
FEATURE IN NAME

Category	Name	Feature
Message	Path.msg	path
	Speed.msg	speed
	RuntimeState.msg	runtime state
Package	graph_msgs	graph
	nav_2d_msgs	navigation 2d
	posedetection_msgs	pose detection

A. Analysis of ROS Message Features

Since the message is written with the feature information, the design solution is to extract the feature of the message and query the desired message based on the message feature. This solution has two benefits compared to using natural language queries. Firstly, it simplifies the developer's input, facilitates querying, and is more in line with the characteristics of this problem. Because developers only know the features for the messages they need to write of the task, like a point, they can just input the feature of the messages to search. Secondly, this approach is easier to implement and use. Because the extraction of features is used on single words, there are already some tools to extract features in words. These tools are also not difficult to use. The time-consuming and difficult processes are collecting raw data and manually processing the data processed by these tools.

The initial collection scheme of message features was to extract the feature from the description text of the message using the natural language processing algorithm [22]. However, two problems were encountered in practice. The first problem is that the message packages are distributed among different repositories, making it difficult to obtain the data. The second problem is that a significant proportion of messages do not have description text, so it is impossible to get the features of these messages from description text.

As shown in Table I, observing and analyzing the message names and message package names, we find there is a lot of feature information in the names. Therefore, we can extract features from the message names and message package names. In addition, since each message will be named, there will not be a problem that there is no source of feature information. We collected all ROS message packages and ROS messages on the ROS website. And next, we verify that the features contained in the names can effectively characterize the message.

We conducted a random sample survey. Since the messages within the same message package have the same code style, only the first message from the message package needs to be sampled. The results are analyzed to represent the features of all messages within that message package. So it is only necessary to take a random sample from 264 message packages and explore the first message within the message package.

TABLE II
PARTIAL SAMPLING RESULTS ABOUT FEATURE

Message Package Name	Message Name	Features	Description of Message
abb_robot_msgs	RAPIDSymbolPath.msg	abb/robot/RAPID/Symbol/Path	#The purpose of this message definition, is to represent **the path** to a **RAPID symbol** (e.g. a variable) defined in an **ABB robot controller** system.
image_exposure_msgs	ImageExposureStatistics.msg	Image/exposure/statistics	# message for **exposure statistics** reported a single **image**
led_msgs	LEDStateArray.msg	led/State/Array	# Full **state** of a **LED strip** or other set of LEDs
warthog_msgs	RGB.msg	warthog/RGB	# Represents the **intensity** of a single **RGB LED**, either reported or commanded.
jackal_msgs	Drive.msg	Jackal/Drive	# This message represents a low-level **motor command** to **Jackal**
pcl_msgs	PolygonMesh.msg	pcl/Polygon/Mesh	# Separate header for the **polygonal surface** # **Vertices** of the **mesh** as a **point cloud** # List of **polygons**

TABLE III
STATISTICAL RESULTS OF THE SAMPLE SURVEY

Category	Number	Percentage
Sampled sample	39	100.0%
Without description	19	48.7%
Fully characterize	13	33.3%
Partially characterize	7	17.9%
Can not characterize	0	0.0%

We randomly selected ten message packages and analyzed them, but half of them had no description text. Therefore, we randomly sampled three more times, ten each time. A total of forty samples were drawn from the four random samples, of which a total of 39 different samples were drawn. The sampling percentage is 14.77% overall. We manually extracted features in message names and message package names and features in description information and compared the extracted features in both. Table II shows some of the sampled messages, the third column is the features manually extracted from the names, and the fourth column is the description text of the message, where the bolded part is the features manually extracted from the description text.

Table III shows the final statistics. It can be seen that the percentage of messages with no description text and messages whose names can fully characterize the features is 82%. There are no messages whose names cannot characterize the message. The analysis of the remaining seven partially characterizable messages reveals that five of them can include the main features of the message. The other two are difficult to represent all the information in the named features because there are many contents, and the text contains much introductory content. However, the name still contains the main features of the message. Therefore, it can be assumed that the features extracted by the message name and the message package name can adequately characterize the message.

B. Data Collection of Knowledge Graph

We collected all message packages and messages from the official ROS website, 264 message packages containing 3244 composite message types. We use a library of English text-splitting modules in python called "wordninja"[8] to extract features from these names automatically. Fig.4 shows an example of word splitting using this module. It can be seen that the module can separate a string of concatenated English words in their entirety.

```
1 import wordninja
2 s = ' thisisausefultool '
3 l = wordninja.splite(s)
4 print(l)
5
6 # Output: ['this', 'is', 'a', 'useful', 'tool']
```

Fig. 4. English text splitting demonstration case.

Initially, more than 9800 feature words were obtained. However, after analyzing these feature words, we found problems such as duplicates, word form variations, noun abbreviations, and single letters. The summary is as follows:

- v+ing: Bounding, Charging, Planning
- v+ed: Stamped, Desired, Planned
- n+s/es: Controls, Trajectories
- single letters: 2, 3, 8, d, etc.
- abbreviations: pos–position, nav–navigation
- prepositions: for, in, at, of, on, with
- be verbs and auxiliary verbs: am/is/are, do
- repetition of feature words

These problems are because there is no standard specification for naming messages. Therefore, secondary data processing is needed for the extracted feature words. And the processing cannot be modified by just designing simple rules. For example, not all words with the suffix ing are verbs, such as string. Completing acronyms is the most difficult part of the whole process. Not only do you need to know the word before the abbreviation, but you also need to determine which word the abbreviation represents based on various aspects of the message. Even if it is the same acronym, it may not be the same word in different message names.

[8]https://pypi.org/project/wordninja/

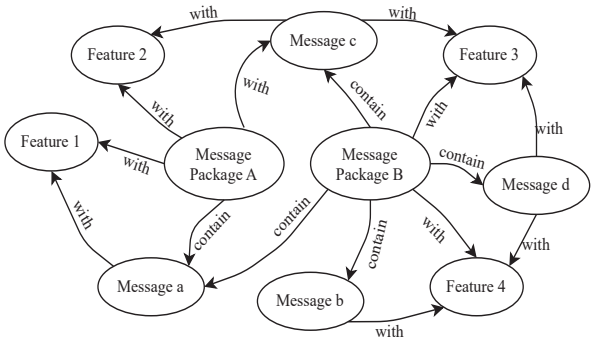

Fig. 5. The relationship between message package, message type, and feature word.

C. Construction of Knowledge Graph

The relationship between messages, message packages, and feature words reveals a complex graph structure among the three. A message package contains multiple messages. Some messages with the same name are contained in different message packages. A message package or a message contains one or more features. And some features will be contained in several different messages and message packages. Fig.5 briefly illustrates a partial relationship diagram between the three. In addition, these message packages and messages have their website describing their contents in detail. Developers need to know the details of the message after finding the desired composite message type. Instead of crawling all these unstructured descriptions and reorganizing them, we take the solution of adding the URLs of these websites into the descriptions of these message packages because it is not only time-consuming to crawl all this information, but also not as good as the display of the original websites to reorganize this information.

In summary, we find the knowledge graph can better solve the search problem. Firstly, the structure of the knowledge graph matches the graph structure between ROS message package, ROS message, and feature words. Secondly, the knowledge graph can add rich information to these graph nodes to help developers know more necessary information. Finally, since there will be some potential semantic and subordination associations between different feature words, the knowledge graph can better discover these associations, thus laying the foundation for the next recommendation function. We choose the Noe4j[9] graph database to build RMKG. When we do entity linking in constructing RMKG, we merge all features to the message package because some feature words come from the message package and do not belong to the message type. Therefore, these features can only be queried by the ROS message package. And in practice, we must know the message package to which the ROS message type belongs, so all features are linked to the message package. Fig.6 shows a part of the structure of RMKG.

[9]https://neo4j.com/

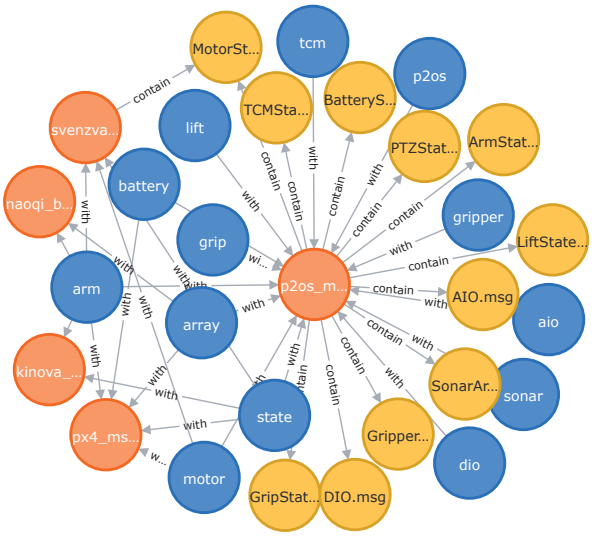

Fig. 6. Subgraphs of RMKG

D. Searching of Required ROS Message

We provide two methods to search for ROS message types. The developers can search ROS message by feature of ROS message or ROS message package. We take shortest path algorithm to search for message in RMKG.

1) Searching by feature: Input the feature f of ROS message and the feature f is contained in feature types set $F \subseteq K$(RMKG), There are ROS message types $m_1, m_2, \ldots, m_n \in K$, ROS package $p_1, p_2, \ldots, p_n \in K$, Function $Min(a, b)$ means the shortest path value between a and b. We calculate the shortest distance $Min(f, m_i), i \in (1, n)$ and select $m_j \in S_1, where\ \forall i, Min(f, m_j) <= Min(f, m_i), i, j \in (1, n)$. The length of all edges defaults to 1. All the message types with the shortest distance to the feature f will be selected to form a set S_1. We rank the message types $m_i \in S_1$ by string similarity and select those message types with a high ranking number. Then all those message packages $p_i \in P, where\ \exists m_i \in S_1, Min(p_i, m_i) = Min(p_i, f) = 1$ form set S_2 . These message types $m_i \in S_1$, message packages $p_i \in S_2$, feature f and those edges between m_i, p_i, f form a subgraph G. This subgraph G contains all possible results of the developer's query because these message types $m_i \in S_1$ include all those message types $m_j \in K$ related to this feature f. Because the feature f is connected to those message packages $p_1, p_2, \ldots, p_j$ that contain the ROS message m_i with the feature f when we build the RMKG. And we will visually display the structural relationship graph G between these message types S_1, message packages S_2, and feature f.

2) Searching by message package: For developers, when searching for ROS message types $m_1, m_2, \ldots, m_n \in K$, if they already know the message package $p_i, where\ \exists m_i, Min(p_i, m_i) = 1, i \in (1, n)$, they can query the ROS message type m_i by entering the message package p_i. Similar

Algorithm 1: Searching by message feature

Input: Feature f
Output: subgraph G

function `SearchMessage(`f`)`**:**
  if $f \in K$ **then**
    $m_i \in G, where, \forall j, Min(m_i, f) <= Min(m_j, f), i, j \in (1, n)$;
    $p_i \in G, where, \exists k, m_k \in G, Min(p_i, m_k) = Min(p_i, f) = 1$;
    $f \in G$;
    $\forall a, b \in G, Min(a, b)_K = 1 \Rightarrow Min(a, b)_G = 1$
      return G;
  end
  return *wrong*;
end

to search by feature, we return those message types $m_i \in K, where\ Min(m_i, p_i) = 1$.

Algorithm 2: Searching by message package

Input: Package p
Output: subgraph G

function `SearchMessage2(`p`)`**:**
  if $p \in K$ **then**
    $m_i \in G, where, \forall j, Min(m_i, p) <= Min(m_j, p), i, j \in (1, n)$;
    $\forall a, b \in G, Min(a, b)_K = 1 \Rightarrow Min(a, b)_G = 1$
      return G;
  end
  return *wrong*;
end

E. Message Package Name Fuzzy Query

However, it is not easy to enter the message package name correctly because the package name is long and complex and contains many acronyms. So it is easy for developers to make a mistake when entering the package name for a query. Although ROS Wiki provides developers with fuzzy search functions for forward and backward matching, users sometimes do not know whether the front and back parts of the package name are long enough in the search process. When the query part is too short, it will return many irrelevant results, and users need to keep flipping many pages to find them.

To solve this problem, we use string similarity ranking. We selected all message package and constructed a message package name library. Since regular expressions cannot solve user input errors, we use string similarity ranking within this message package library to query the message package name. The string similarity calculation method uses the least edit distance [23] and containment matching algorithms. The minimum edit distance is the minimum number of edit operations required to convert from one to the other between two strings. Permissioned editing operations include replacing one character with another, inserting a character, and deleting a character. In general, the smaller the edit distance, the more similar the two strings are. We use a module in python called "fuzzywuzzy" to implement string similarity ranking[18]. Using this method, developers need not input all the package names and input a short part of the whole name. They also can input the wrong name within a certain wrong character amount. So, developers no longer need to remember the complex package name.

V. Experimental Evaluation

For the following three research questions, we designed experiments to evaluate them.

- **RQ1**: How is the intrinsic quality of the knowledge captured in the constructed RMKG?
- **RQ2**: How does RMKG perform in ROS message type searching task compared with baseline approach?
- **RQ3**: How does the message package fuzzy query approach perform in package query tasks?

A. The Quality of RMKG.

The knowledge is extracted from two main sources: ROS messages and ROS message packages. As we use the 'wordninja' tool to extract the features from messages and message packages information, we mainly evaluate the accuracy of the knowledge extraction from messages and message packages (i.e., feature extraction, entities linking) [21] [24].

Since the method uses message features to query ROS message types and the extracted knowledge are features, the metric for assessing the quality of the knowledge is whether the features are correctly extracted from messages and message packages and whether features extracted from message A fully include all features of ROS message type A [21]. Because we already manually corrected all the feature data that we use the 'wordninja' tool extract from messages and message packages, so we evaluate whether features extracted from message A fully include all features of ROS message type A (Message A does not refer to a specific message, but refers to all messages in general).

So we will manually extract the features from messages and calculate whether the features extracted by 'wordninja' from a certain message include all the features manually extracted from the same message of us.

1) Protocol: Similar to previous studies [21] [25]–[27], we used a random sampling method [28], and with a confidence level of 95% and a confidence interval of 5, we randomly drew 344 samples from the overall 3,244 message types. One

TABLE IV
SAMPLING STATISTICS RESULTS OF RMKG QUALITY

Category	Number	Percentage
Characterize all features	319	92.73%
Characterize some features	14	4.07%
Not characterize	11	3.20%

developer(with one and a half years of ROS development experience) independently performs the examination.

2) Result and Analysis: The results are shown in Table IV. It is clear that 92.73% of the message types completely extracted all the features, so the mapping is of high quality. The quality of RMKG and the accuracy of the knowledge extraction from messages and message packages are high.

Analysis of the negative results shows that the reason for characterizing only some of the features of the messages is that the feature words were not extracted correctly by the English word segmentation tool and the message names contain acronyms with no obvious semantic meaning. Message names that do not characterize the message are usually composed of only acronyms with no obvious semantic meaning, and these acronyms are removed during second data cleaning.

Because all the features extracted from certain messages are linked to the same message and all messages are linked to the packages, entities linking have 100% accuracy [21].

> *Our ROS message features extraction methods for constructing ROS message knowledge graph are accurate, which can support practical use.*

B. Validity Assessment of Our Approach

We evaluate the usefulness of RMKG Approach(RMKGA) in ROS message type searching tasks, that is, finding the suitable ROS message type for specific data transmission tasks.

1) Task: Since the role of the message is to transmit the communication data between nodes, we set 10 different common data transmission tasks to allow the experimenter to search for the message data type that can satisfy the data transmission task. Those seldom-used ROS message types are very hard to search by general search engines such as Baidu because there are few related blogs or Q&A content. But they can be searched by our approach based on RMKG because we collect all ROS message types. So if RMKG Approach(RMKGA) is better than the baseline approach on common ROS message types, then RMKGA is better on seldom-used message types. The 10 different common data transmission tasks are transmitting the messages of color, image, 3D point, pose, 3D vector, path, accel data, speed, twist, distance.

2) Baseline: We use ROS Wiki and Baidu search engine as the baseline tool. For ROS-based robotics development, ROS Wiki is the most commonly used knowledge search community. Baidu is the most widely used Chinese search engine and the most used by Chinese developers, which contains a large number of technical blogs and Q&A community content.

3) Protocol: We recruit 3 master students from our school and all of them have ROS-based robotics software development experience. We believe these students are qualified for our study. For every task, the experimenters need to search at least one messages that satisfy the data transmission task we pre-set, using the baseline approach and RMKGA respectively. We will manually evaluate whether the search results meet the task, and only the results that meet the task are considered to be successfully searched. For each task, we count the spending time and the number of search results related to the task.

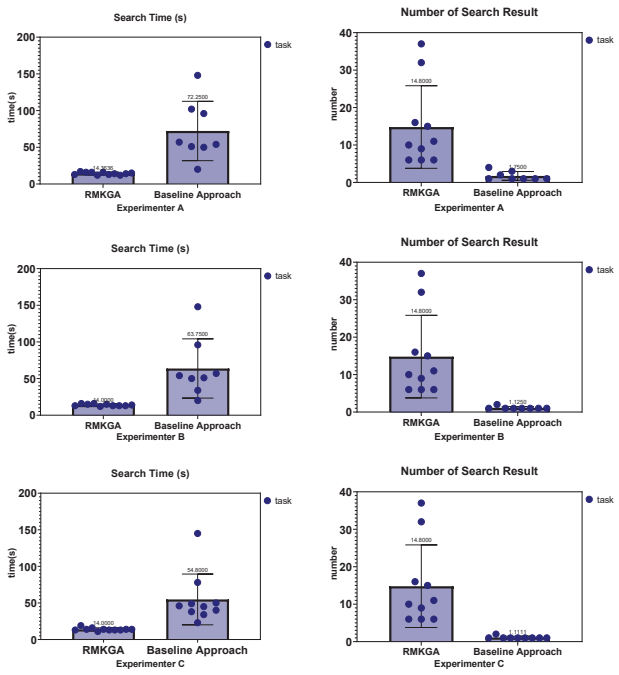

Fig. 7. Comparison of Search Efficiency

4) Result and Analysis: Fig7 shows the completion time and the search result number of each task by RMKGA and baseline approach for 3 experimenters. The average search time for 10 tasks of 3 experimenters by RMKGA is 14.1 seconds that are less than a quarter of the average search time of 63.6 seconds by the baseline approach. And there are two tasks in which all the experimenters fail to find the result by the baseline approach. The success rate of baseline approach on common ROS message types is 80%. All 10 tasks are successfully finished by the RMKGA. The success rate of RMKGA on common ROS message types is 100%. Completion times using the baseline approach are more spread out because the amount of related content varies across different ROS message types and there are many unrelated contents searched, which causes some ROS message types are hard to find and some ROS message types are relatively easy to find. Because the RMKG approach uses feature words to search for related ROS message types, so as long as input correct feature words, RMKG will return related results with no irrelevant content. So the completion time is concentrated. Using the RMKGA can find 14.8 relevant results per task on average, while the baseline approach can find only 1.3 results per task on average.

The reason for this result is that the Baidu search engine can only search for some technical blogs about ROS and ROS official website. These blogs only contain a small part of ROS message types, and other ROS message types that no one introduces cannot be searched. In addition, it is not possible to

TABLE V
FUZZY QUERY EFFECT OF PACKAGE

Package	Input	Result	Rank
geometry_msgs	geometry	√	1
	gomtry	√	1
	gmt	√	8
	gry	√	4
	geo	√	2
	try	√	1
abb_rapid_sm_addin_msgs	abb_rapid	√	2
	abrpd	√	2
	abr	√	3
	abb	√	3
	sm	√	1
	admin	√	1
automative_navigation_msgs	automative	√	1
	auto	√	1
	aotu	√	11
	aotunvgt	√	1
	autonav	√	8
	navigation	√	1
nav_2d_msgs	nav_2d	√	1
	navi	√	6
	nav	√	5
	nv	×	
	nv2	√	6
	2d	√	2

search for ROS message types directly on the ROS wiki. The RMKG collects all the ROS message types officially included by ROS, so developers can find all ROS message types and all message packages. During the experiment, the experimenter often cannot find the result by searching once when using the Baidu search engine and need to continuously enrich and modify query statements.

Although by no means conclusive due to the small-scale of our study, our pilot user study demonstrates that our approach significantly decreases the time of ROS message search tasks for developers.

C. Fuzzy Query Effect Evaluation

We evaluate the effect of our fuzzy query method based on string similarity ranking in ROS package query tasks that are inputting different query words under five situations for four query tasks.

1) Protocol: We select four packages that are complex and long. For every package, we input six different search words with five situations that input long prefix of the name, the short prefix of the name, short infix of name, wrong part of the name, and mixed combination of the name. There are twenty search results every time for inputting the words. We observe whether the target package is returned. If it returns the search package, we mark the result as true, else false. And we count the rank of the search package in the twenty results when the search package is returned. The higher the ranking is, the better the effect of fuzzy search.

2) Result: Table V shows that there is only one input word with no returning right result in a total of 24 input words. There are ten times with the rank number of one, 15 times with rank number higher than 4, 17 times with rank number higher than 6, and 22 times with rank number higher than 9. We find all the long and short prefixes, and an infix of the name can search out the target package with a high-rank number. The less wrong input of name also can return target package with a high-rank number. But the mixed combination of names performs not so well. Although the mixed combination of names can return the correct result with high probability, the rank is low. So our method of fuzzy query based on string similarity ranking on this problem performs well.

3) Analysis: The reasons for causing these experimental results are that the package names have their special letter and words combination structure, and the package library is designed for special questions on a small scale. So as long as the input word contains some structural information of the target package, the target package can be found, even if the input word contains some errors.

Our ROS package fuzzy search method based on string similarity ranking is effective under the above five input situations, which can support practical use and solve the problem of the difficult inputting package names

VI. VALIDITY THREATS

One of the main threats against RMKG is in the feature processing phase. Since the acronym completion is more dependent on the understanding of the message, there may be some cases where the acronym completion is incorrect. In addition, there may be a small number of acronyms that are not detected and thus not complemented. But the number of these missing acronyms is relatively small. Moreover, there are a small number of messages for which feature information cannot be extracted, thus making these message types difficult to be searched.

The fuzzy matching of package names based on string similarity works well when the number of packages is not very large. However, suppose the number of packages reaches hundreds of thousands or even more, there may be many similar package names, so that string similarity matching will match many similarly-named package names and fail to return the correct package that the user is looking for. However, based on the current development of ROS, this situation will not occur yet.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose an approach to search message types by inputting message features on RMKG to address the problem that ROS developers have difficulty finding the required composite message types based on the current task requirements. The evaluation results show that the quality of RMKG is good. For this problem, the method is efficient. Also, the experiments show that extracting feature information by message name and message package name is effective. For another problem that it isn't easy to input the message package correctly when querying ROS message types by

message package, we propose a method to query the message package based on string similarity ranking. The evaluation results show that the method is effective. In the future, we will establish the association between message feature words to recommend some highly relevant composite message types when developers search for them to provide more references to developers. And we will improve the recommendation probability of commonly used composite message types in the recommendation engine.

ACKNOWLEDGMENT

This work was supported by the Key Laboratory of Software Engineering for Complex Systems and the National Science Foundation of China under granted number 62172426.

REFERENCES

[1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, *et al.*, "Ros: an open-source robot operating system," in *ICRA workshop on open source software*, vol. 3, no. 3.2. Kobe, Japan, 2009, p. 5.

[2] X. Wang, X. Liu, J. Liu, X. Chen, and H. Wu, "A novel knowledge graph embedding based api recommendation method for mashup development," *World Wide Web*, vol. 24, no. 3, pp. 869–894, 2021.

[3] H. Yin, Y. Zheng, Y. Sun, and G. Huang, "An api learning service for inexperienced developers based on api knowledge graph," in *2021 IEEE International Conference on Web Services (ICWS)*. IEEE, 2021, pp. 251–261.

[4] A. T. Nguyen, M. Hilton, M. Codoban, H. A. Nguyen, L. Mast, E. Rademacher, T. N. Nguyen, and D. Dig, "Api code recommendation using statistical learning from fine-grained changes," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 511–522.

[5] Q. Huang, X. Xia, Z. Xing, D. Lo, and X. Wang, "Api method recommendation without worrying about the task-api knowledge gap," in *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2018, pp. 293–304.

[6] S. Gao, L. Liu, Y. Liu, H. Liu, and Y. Wang, "Api recommendation for the development of android app features based on the knowledge mined from app stores," *Science of Computer Programming*, vol. 202, p. 102556, 2021.

[7] F. Thung, "Api recommendation system for software development," in *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2016, pp. 896–899.

[8] M. M. Rahman, C. K. Roy, and D. Lo, "Rack: Automatic api recommendation using crowdsourced knowledge," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 1. IEEE, 2016, pp. 349–359.

[9] X. Liu, L. Huang, and V. Ng, "Effective api recommendation without historical software repositories," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 282–292.

[10] C. Chen, X. Peng, Z. Xing, J. Sun, X. Wang, Y. Zhao, and W. Zhao, "Holistic combination of structural and textual code information for context based api recommendation," *IEEE Transactions on Software Engineering*, 2021.

[11] W. Yuan, H. H. Nguyen, L. Jiang, and Y. Chen, "Libraryguru: Api recommendation for android developers," in *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, 2018, pp. 364–365.

[12] X. He, L. Xu, X. Zhang, R. Hao, Y. Feng, and B. Xu, "Pyart: Python api recommendation in real-time," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1634–1645.

[13] P. T. Nguyen, J. Di Rocco, C. Di Sipio, D. Di Ruscio, and M. Di Penta, "Recommending api function calls and code snippets to support software development," *IEEE Transactions on Software Engineering*, 2021.

[14] A. Alnusair, T. Zhao, and E. Bodden, "Effective api navigation and reuse," in *2010 IEEE International Conference on Information Reuse & Integration*. IEEE, 2010, pp. 7–12.

[15] J. Wu, L. Shen, W. Guo, and W. Zhao, "Code recommendation for android development: how does it work and what can be improved?" *Science China Information Sciences*, vol. 60, no. 9, pp. 1–14, 2017.

[16] S. Luan, D. Yang, C. Barnaby, K. Sen, and S. Chandra, "Aroma: Code recommendation via structural code search," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 1–28, 2019.

[17] S. Wang, M. Wen, B. Lin, and X. Mao, "Lightweight global and local contexts guided method name recommendation with prior knowledge," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 741–753.

[18] N. Murakami and H. Masuhara, "Optimizing a search-based code recommendation system," in *2012 Third International Workshop on Recommendation Systems for Software Engineering (RSSE)*. IEEE, 2012, pp. 68–72.

[19] S. Ji, S. Pan, E. Cambria, P. Marttinen, and S. Y. Philip, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Transactions on Neural Networks and Learning Systems*, 2021.

[20] S. Zamanirad, B. Benatallah, M. C. Barukh, F. Casati, and C. Rodriguez, "Programming bots by synthesizing natural language expressions into api invocations," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2017, pp. 832–837.

[21] L. Chen, X. Mao, Y. Zhang, S. Yang, and S. Wang, "An efficient ros package searching approach powered by knowledge graph," *Proceedings of the International Conference on Software Engineering and Knowledge Engineering*, pp. 411–416, 2021.

[22] X. Zhao, Z. Xing, M. A. Kabir, N. Sawada, J. Li, and S.-W. Lin, "Hdskg: Harvesting domain specific knowledge graph from content of webpages," in *2017 ieee 24th international conference on software analysis, evolution and reengineering (saner)*. IEEE, 2017, pp. 56–67.

[23] Y. Zhao, H. Jiang, and X. Wang, "Minimum edit distance-based text matching algorithm," in *Proceedings of the 6th International Conference on Natural Language Processing and Knowledge Engineering (NLPKE-2010)*. IEEE, 2010, pp. 1–4.

[24] H. Chen, G. Cao, J. Chen, and J. Ding, "A practical framework for evaluating the quality of knowledge graph," in *China conference on knowledge graph and semantic computing*. Springer, 2019, pp. 111–122.

[25] H. Li, S. Li, J. Sun, Z. Xing, X. Peng, M. Liu, and X. Zhao, "Improving api caveats accessibility by mining api caveats knowledge graph," in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2018, pp. 183–193.

[26] X. Ren, X. Ye, Z. Xing, X. Xia, X. Xu, L. Zhu, and J. Sun, "Api-misuse detection driven by fine-grained api-constraint knowledge graph," in *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2020, pp. 461–472.

[27] C. Wang, X. Peng, M. Liu, Z. Xing, X. Bai, B. Xie, and T. Wang, "A learning-based approach for automatic construction of domain glossary from source code and documentation," in *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 97–108.

[28] R. Singh and N. S. Mangat, *Elements of survey sampling*. Springer Science & Business Media, 2013, vol. 15.